

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

REPORT

Laboratory work No. 10

Add Your Own System Call: trace

Rotaru Anastasia, FAF-242

Course: **Operating Systems**

Checked by: Patricia Reiţman, asist. univ.

Chişinău, 2026

Contents

1 Goal of the work **2**

2 Part 1. The number and the user side **2**
2.1 Changes . . . 2
2.2 Check 1 . . . 3

3 Part 2. The kernel side **3**
3.1 Changes . . . 3
3.2 Problems encountered . . . 4
3.3 Check 2 . . . 4

4 Part 3. Playing with the mask **4**
4.1 All calls . . . 4
4.2 Only `fork` . . . 5
4.3 Only `write` . . . 5
4.4 Only `open` and `close` . . . 5
4.5 Observations . . . 5

5 Conclusion **6**

1 Goal of the work

The goal of this laboratory assignment is to add a new system call to the xv6 kernel. The new call, `trace`, makes a process print every system call it makes, similar to what `strace` did in Laboratory 9 on Linux. The work covers the number assigned to the call, the user-side stub and program, the kernel handler, and the per-process mask that decides which calls are printed. The work was performed in an Ubuntu 24.04 (arm64) container running on Docker Desktop on macOS, with QEMU 8.2.2 and the `riscv64-unknown-elf-gcc` 13.2.0 toolchain.

2 Part 1. The number and the user side

2.1 Changes

The system call number `SYS_trace` is defined as 23 in `kernel/syscall.h`. The number 22 was already used by `sync` in this tree, so the value 22 could not be reused.

```
#define SYS_sync  22
#define SYS_trace 23
```

The declaration is added to `user/user.h`, and the assembly stub is generated by `user/usys.pl`:

```
int trace(int);          // user/user.h
```

```
entry("trace");          # user/usys.pl
```

The user program `user/trace.c` sets the mask on the calling process and then replaces the process image with the requested command. The mask survives `exec`, so the command is traced from its first call:

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if (argc < 3) {
    fprintf(2, "usage: trace mask command [args...]\n");
    exit(1);
  }
  if (trace(atoi(argv[1])) < 0) {
    fprintf(2, "trace: failed\n");
    exit(1);
  }
  exec(argv[2], argv + 2);
  fprintf(2, "trace: exec %s failed\n", argv[2]);
  exit(1);
}
```

Finally, `$U/_trace\` is added to the `UPROGS` list in the `Makefile`, after `$U/_zombie\`, so that the program is placed on the xv6 disk image.

2.2 Check 1

Before the kernel side existed, the number 23 had no handler. The expected result was the kernel's complaint about an unknown call, which shows that the number reaches the kernel:

```
$ trace 32 grep hello README
3 trace: unknown sys call 23
trace: failed
```

3 Part 2. The kernel side

3.1 Changes

Each process stores its own mask. A field is added to `struct proc` in `kernel/proc.h`, after `name`:

```
struct proc {
  ...
  struct inode *cwd;           // Current directory
  char name[16];               // Process name (debugging)
  int trace_mask;              // which system calls to print
};
```

The handler `sys_trace` in `kernel/sysproc.c` reads the first argument and stores it in the current process:

```
uint64
sys_trace(void)
{
  int mask;
  argint(0, &mask);
  myproc()->trace_mask = mask;
  return 0;
}
```

In `kernel/syscall.c`, the handler is declared and placed in the dispatch table, and a table of names is added so that the output can show the name of each call:

```
extern uint64 sys_trace(void);
...
  [SYS_trace]   = sys_trace,
```

```
static char *syscallnames[] = {
  [SYS_fork]   = "fork",
  [SYS_read]   = "read",
  ...
  [SYS_sync]   = "sync",
```

```
  [SYS_trace]  = "trace",
};
```

The printing happens in `syscall()`, after the call has returned, so that the return value is already in `a0`:

```
p->trapframe->a0 = syscalls[num]();
if ((p->trace_mask >> num) \& 1)
  printk("%d: syscall %s -> %d\n",
         p->pid, syscallnames[num], (int)p->trapframe->a0);
```

Children inherit the mask. In `kfork()` in `kernel/proc.c`, the mask is copied right after the name:

```
safestrcpy(np->name, p->name, sizeof(p->name));
np->trace_mask = p->trace_mask;
```

3.2 Problems encountered

While editing `struct proc`, stray lines were left behind: a duplicated closing brace `};` and a comment copied onto the wrong field. The compiler reported `expected identifier or '(' before '}' token` for `kernel/proc.h`, and no kernel could be built until the structure was corrected to the version shown above. The `bc: not found` and `[: Illegal number` messages printed by `make` are produced by the Makefile and do not affect the build.

3.3 Check 2

With the kernel side complete, the call prints one line for each `read` made by `grep`. The byte counts add up to the size of `README`, which is 2441 bytes ($1023 + 965 + 453 = 2441$), followed by 0 at the end of the file:

```
$ trace 32 grep hello README
3: syscall read -> 1023
3: syscall read -> 965
3: syscall read -> 453
3: syscall read -> 0
```

4 Part 3. Playing with the mask

The mask is read in binary: bit N set means that system call number N is printed. For example, $32 = 2^5$, and system call 5 is `read`.

4.1 All calls

With all bits set, every system call of the traced process is printed, including `trace` itself:

```
$ trace 2147483647 grep hello README
3: syscall trace -> 0
3: syscall exec -> 3
```

```
3: syscall open -> 3
3: syscall read -> 1023
3: syscall read -> 965
3: syscall read -> 453
3: syscall read -> 0
3: syscall close -> 0
```

4.2 Only `fork`

Mask 2 selects bit 1, which is `fork`. The test `forkforkfork` creates processes until the process table is full, and each line shows the pid of the process that called `fork`. The beginning and the end of the output are shown below, with the middle omitted:

```
$ trace 2 usertests forkforkfork
usertests starting
test forkforkfork: 4: syscall fork -> 5
5: syscall fork -> 6
6: syscall fork -> 7
...
6: syscall fork -> -1
8: syscall fork -> -1
OK
ALL TESTS PASSED
```

4.3 Only `write`

Mask 65536 selects bit 16, which is `write`:

```
$ trace 65536 echo hi
hi66: syscall write -> 2

66: syscall write -> 1
```

4.4 Only `open` and `close`

The mask that selects bits 15 and 21 is computed as:

$$\text{mask} = 2^{15} + 2^{21} = 32768 + 2097152 = 2129920$$

```
$ trace 2129920 grep hello README
5: syscall open -> 3
5: syscall close -> 0
```

4.5 Observations

1. **Why only `read` appears for mask 32:** Mask 32 is 2^5, so only bit 5 is set. Bit 5 is `SYS_read`. The kernel checks `(p->trace_mask >> num) & 1` for each call, so `open` (15) and `close` (21) are not printed.

2. **Why the children print lines:** The processes created by `forkforkfork` have different pids, and each one prints its own `fork` calls. The line `np->trace_mask = p->trace_mask;` in `kfork()` copies the parent's mask into the child. Without that line, children would have mask 0 and print nothing.

3. **Why the trace line appears between `hi` and the newline:** `echo` makes two `write` calls: one for `"hi"` and one for the newline. The kernel prints the trace line right after the call returns. The trace line for the first call therefore appears after `hi` and before the second `write` of the newline.

4. **The mask for `open` and `close`:** Setting bits 15 and 21 gives $2^{15} + 2^{21} = 2129920$. Running `trace 2129920 grep hello README` prints only the two calls.

5 Conclusion

In this laboratory work I added a new system call to xv6. The number 23 is passed from a user program through the `ecall` stub into the kernel, where a handler stores a per-process mask. The mask is copied to children in `fork`, and after each call the kernel prints the call if its bit is set. The experiments showed how the mask selects calls, and that a mask written as a sum of powers of two selects exactly those calls.